

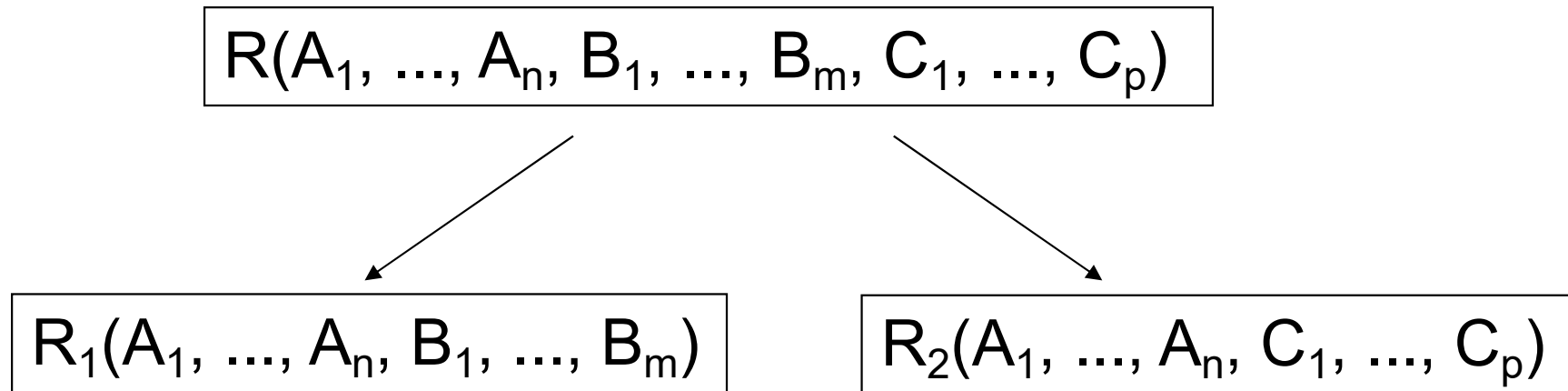
Outline

- Problems Caused by Redundancy
- Functional Dependencies (FDs)
- Normal Forms Based on Keys
- **Decomposition**
 - **Lossless Join Decomposition**
 - Dependency Preserving Decomposition
- Normalization
 - BCNF Decomposition Algorithm
 - 3NF Decomposition Algorithm

Decomposition

- Decomposition is a tool that allows us to eliminate redundancy
- It is important to check that a decomposition does not introduce new problems
 - A decomposition allows us to recover the original relation?
 - Can we check integrity constraints efficiently?

Projection for Decomposition



R_1 = projection of R on $A_1, \dots, A_n, B_1, \dots, B_m$

R_2 = projection of R on $A_1, \dots, A_n, C_1, \dots, C_p$

$A_1, \dots, A_n \cup B_1, \dots, B_m \cup C_1, \dots, C_p$ = all attributes

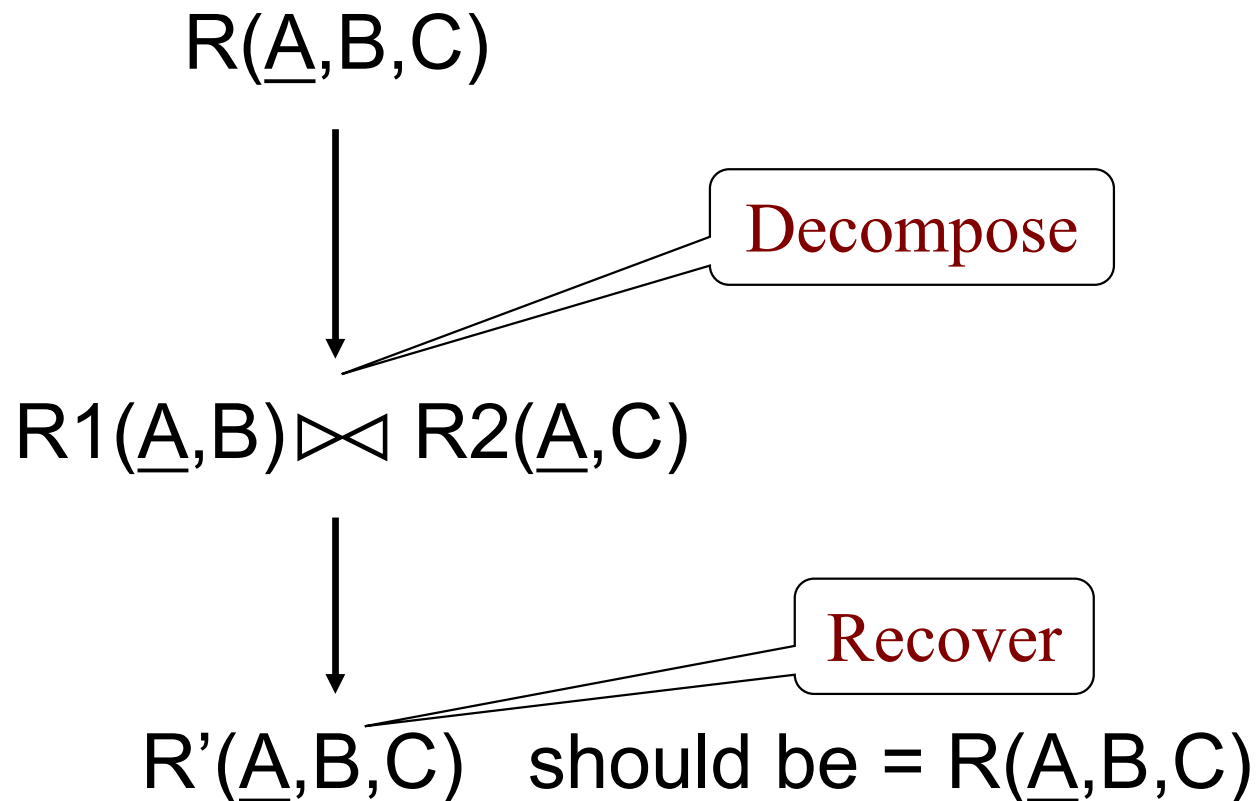
R_1 and R_2 may (not) be reassembled to produce original R

Problems with Decomposition

- Given instances of the decomposed relations, we may not be able to reconstruct the corresponding instance of the original relation – information loss
- Checking some dependencies may require joining the instances of the decomposed relations

Lossless Join Decompositions

A decomposition is *lossless* if we can recover:

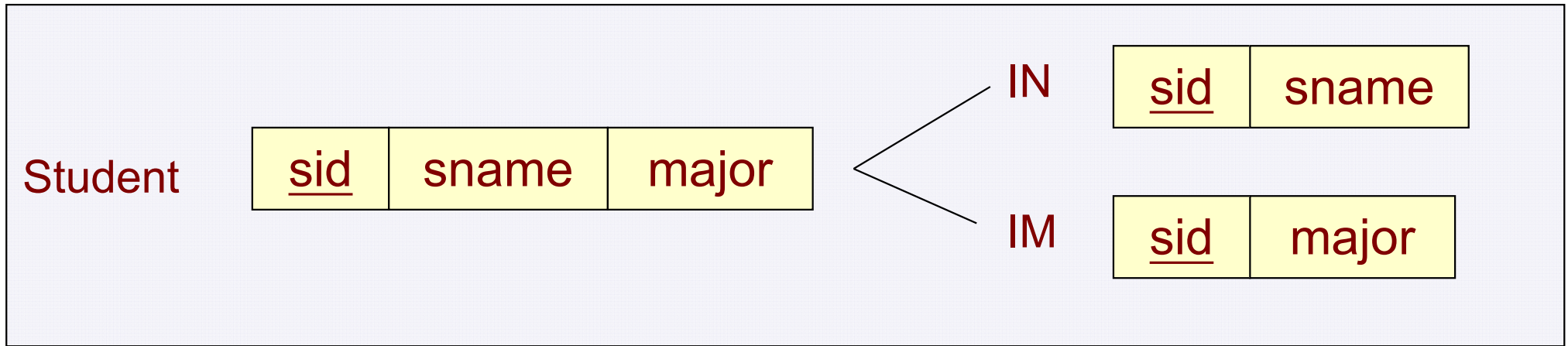


R' is in general larger than R . Must ensure $R' = R$

Lossless Join Decomposition

- $\{ R_1, \dots, R_n \}$ – a set of relation schemas
- $\{ R_1, \dots, R_n \}$ is a *decomposition* of R if $R_1 \cup R_2 \cup \dots \cup R_n = R$
- $\{ R_1, \dots, R_n \}$ is a *lossless-join decomposition* of R if, for all relations r on schema R , $\Pi_{R_1}(r) \bowtie \dots \bowtie \Pi_{R_n}(r) = r$

Example: A Lossless Join Decomposition



Student

<u>sid</u>	sname	major
123	Ling Wang	C.S.
456	Hong Zhou	C.S.

IN

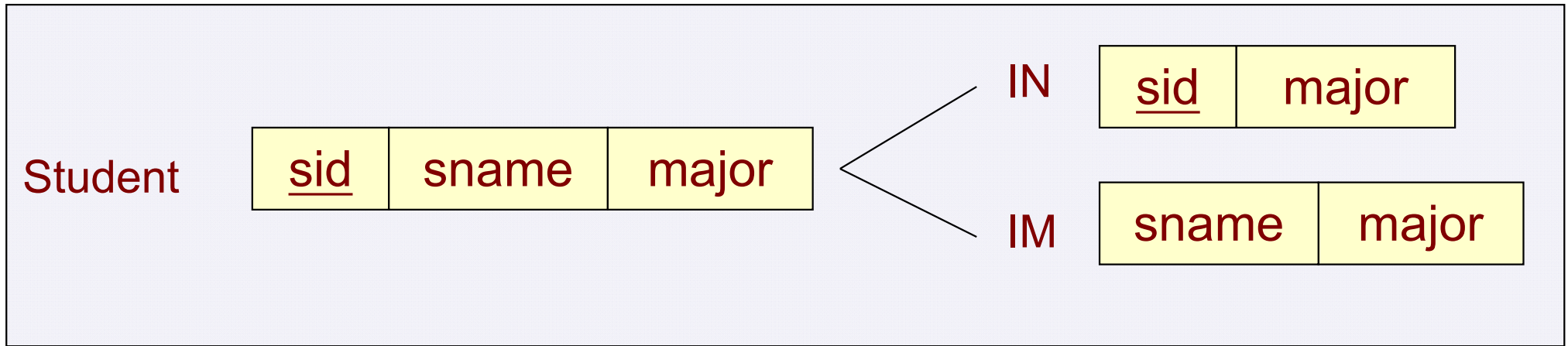
<u>sid</u>	sname
123	Ling Wang
456	Hong Zhou

'Student' can be recovered by joining the instances of IN and IM

IM

<u>sid</u>	major
123	C.S.
456	C.S.

Example: A Non-Lossless Join Decomposition



Student

<u>sid</u>	sname	major
123	Ling Wang	C.S.
456	Hong Zhou	C.S.

IN

<u>sid</u>	major
123	C.S.
456	C.S.

Student = IN ⋈ IM????

IM

sname	major
Ling Wang	C.S.
Hong Zhou	C.S.

Example: A Non-Lossless Join Decomposition

IN

<u>sid</u>	major
123	C.S.
456	C.S.

IM

sname	major
Ling Wang	C.S.
Hong Zhou	C.S.

IN $\bowtie$ IM

sid	sname	major
123	Ling Wang	C.S.
123	Hong Zhou	C.S.
456	Ling Wang	C.S.
456	Hong Zhou	C.S.

$\neq$

Student

<u>sid</u>	sname	major
123	Ling Wang	C.S.
456	Hong Zhou	C.S.

The instance of 'Student' cannot be recovered by joining the instances of IM and NM. Therefore, such a decomposition is not a lossless join decomposition

Theorem: Lossless Join Decomposition

- R – a relation schema
- F – a set of functional dependencies on R
- $\{ R_1, R_2 \}$ is a lossless-join decomposition of R

$$\Leftrightarrow (R_1 \cap R_2) \rightarrow R_1 \in F^+ \text{ or } (R_1 \cap R_2) \rightarrow R_2 \in F^+$$

$$\Leftrightarrow (R_1 \cap R_2) \text{ is a } \textit{key} \text{ for } R_1 \text{ or } R_2$$

- (the attributes common to R_1 and R_2 must contain a key for either R_1 or R_2)

Example: Non-Lossless Join

$F = \{ B \rightarrow C, C \rightarrow B \}$

$A \rightarrow AB \in F^+?$ no

$A \rightarrow AC \in F^+?$ no

Decompose

A	B	C
a ₁	b ₁	c ₁
a ₁	b ₂	c ₂

A	B
a ₁	b ₁
a ₁	b ₂

A	C
a ₁	c ₁
a ₁	c ₂

Join

A	B	C
a ₁	b ₁	c ₁
a ₁	b ₁	c ₂
a ₁	b ₂	c ₁
a ₁	b ₂	c ₂

Example: Lossless Join

$F = \{ B \rightarrow C, C \rightarrow B \}$

$C \rightarrow AC \in F^+?$ no

$C \rightarrow BC \in F^+?$ yes

Decompose

A	B	C
a ₁	b ₁	c ₁
a ₁	b ₂	c ₂

A	C
a ₁	c ₁
a ₁	c ₂

B	C
b ₁	c ₁
b ₂	c ₂

Join

A	B	C
a ₁	b ₁	c ₁
a ₁	b ₂	c ₂

Example: Lossless Join Decomposition

- $R = (A, B, C)$
- $F = \{A \rightarrow B\}$
- $\{A, B\} + \{A, C\}$ is a lossless join decomposition
- $\{A, B\} \cap \{A, C\} = \{A\}$ is a key in $\{A, B\}$
- $\{A, B\} + \{B, C\}$ is not a lossless join decomposition
- $\{A, B\} \cap \{B, C\} = \{B\}$ is neither a key in $\{A, B\}$ nor $\{B, C\}$

Outline

- Problems Caused by Redundancy
- Functional Dependencies (FDs)
- Normal Forms Based on Keys
- **Decomposition**
 - Lossless Join Decomposition
 - **Dependency Preserving Decomposition**
- Normalization
 - BCNF Decomposition Algorithm
 - 3NF Decomposition Algorithm

Dependency Preservation

- R – a relation schema
- F – a set of functional dependencies on R
- $\{R_1, R_2\}$ is a decomposition of R
- F_i – a subset of F with only attributes in R_i
(i.e. the *projection* of F on R_i)

- Dependency is *preserved* if $(F_1 \cup F_2)^+ = F^+$

e.g.: $R = (A, B, C)$, $F = \{A \rightarrow B, B \rightarrow C\}$

$R_1 = (A, B)$ and $R_2 = (B, C)$

$F_1 = \{A \rightarrow B\}$ and $F_2 = \{B \rightarrow C\}$

Dependency Preserving Decomposition

- E.g., $R = (A, B, C)$, $F = \{A \rightarrow B, C \rightarrow B\}$

$$R_1 = (A, B) \text{ and } R_2 = (A, C)$$

$$F_1 = \{A \rightarrow B\} \text{ and } F_2 = \{\}$$

$C \rightarrow B$ is not in both F_1 and F_2

- If dependency is **NOT** preserved, *joins must be computed* in order to check if an update is illegal
- Very inefficient ... !!!

Example: Dependency Preservation

Dependency set: $F = \{ \text{sid} \rightarrow \text{dname}, \text{dname} \rightarrow \text{dhead} \}$

Student

<u>sid</u>	dname	dhead
------------	-------	-------

$F = \{ \text{sid} \rightarrow \text{dname}, \text{dname} \rightarrow \text{dhead} \}$

decomposition

IN

<u>sid</u>	dname
------------	-------

$F_{IN} = \{ \text{sid} \rightarrow \text{dname} \}$

IH

<u>sid</u>	dhead
------------	-------

$F_{IH} = \{ \text{sid} \rightarrow \text{dhead} \}$

IN	<table border="1"><tr><td><u>sid</u></td><td>dname</td></tr></table>	<u>sid</u>	dname
<u>sid</u>	dname		

IH	<table border="1"><tr><td><u>sid</u></td><td>dhead</td></tr></table>	<u>sid</u>	dhead
<u>sid</u>	dhead		

Dependency set: $F = \{ \text{sid} \rightarrow \text{dname}, \text{dname} \rightarrow \text{dhead} \}$

$$F_{IN} = \{ \text{sid} \rightarrow \text{dname} \}$$

$$F_{IH} = \{ \text{sid} \rightarrow \text{dhead} \}$$

Dependency Preserving?

- $(\text{dname} \rightarrow \text{dhead}) \in F^+$ but
- $(\text{dname} \rightarrow \text{dhead}) \notin (F_{IN} \cup F_{IH})^+$

This decomposition **does not** preserve dependency

IN	<u>sid</u>	dname
----	------------	-------

IH	<u>sid</u>	dhead
----	------------	-------

Dependency set: $F = \{ \text{sid} \rightarrow \text{dname}, \text{dname} \rightarrow \text{dhead} \}$

Student	<u>sid</u>	dname	dhead
	123	C.S.	Ying Wang
	456	C.S.	Ying Wang

$F_{IN} = \{ \text{sid} \rightarrow \text{dname} \}$

IN	<u>sid</u>	dname
	123	C.S.
	456	C.S.

$F_{IH} = \{ \text{sid} \rightarrow \text{dhead} \}$

IH	<u>sid</u>	dhead
	123	Ying Wang
	456	Ying Wang

The update violates the FD

' $\text{dname} \rightarrow \text{dhead}$ '

However, it can only be caught
when we join IN and IH



Updated to

<u>sid</u>	dhead
123	Ying Wang
456	Lin Cheung

Example: Dependency Preservation

Dependency set: $F = \{ \text{sid} \rightarrow \text{dname}, \text{dname} \rightarrow \text{dhead} \}$

Let's decompose the relation in another way

Student

<u>sid</u>	dname	dhead
------------	-------	-------

$F = \{ \text{sid} \rightarrow \text{dname}, \text{dname} \rightarrow \text{dhead} \}$

decomposition

IN

<u>sid</u>	dname
------------	-------

$F_{IN} = \{ \text{sid} \rightarrow \text{dname} \}$

NH

dname	dhead
-------	-------

$F_{NH} = \{ \text{dname} \rightarrow \text{dhead} \}$

IN	<table border="1"><tr><td><u>sid</u></td><td>dname</td></tr></table>	<u>sid</u>	dname
<u>sid</u>	dname		

NH	<table border="1"><tr><td>dname</td><td>dhead</td></tr></table>	dname	dhead
dname	dhead		

Dependency set: $F = \{ \text{sid} \rightarrow \text{dname}, \text{dname} \rightarrow \text{dhead} \}$

$$F_{IN} = \{ \text{sid} \rightarrow \text{dname} \}$$

$$F_{NH} = \{ \text{dname} \rightarrow \text{dhead} \}$$

Dependency Preserving?

- $(F_{IN} \cup F_{NH})^+ = F^+$

This decomposition preserve dependency

IN	<u>sid</u>	dname
----	------------	-------

NH	dname	dhead
----	-------	-------

Dependency set: $F = \{ \text{sid} \rightarrow \text{dname}, \text{dname} \rightarrow \text{dhead} \}$

Student	<u>sid</u>	dname	dhead
	123	C.S.	Ying Wang
	456	C.S.	Ying Wang

IN	<u>sid</u>	dname
	123	C.S.
	456	C.S.

NH	<u>dname</u>	dhead
	C.S.	Ying Wang
	C.S.	Ying Wang



Updated to

	<u>dname</u>	dhead
	C.S.	Ying Wang
	C.S.	Lin Cheung

The error in NH will immediately be caught by the DBMS,
since it violates F.D. $\text{dname} \rightarrow \text{dhead}$
No join is necessary for the detection

Outline

- Problems Caused by Redundancy
- Functional Dependencies (FDs)
- Normal Forms Based on Keys
- Decomposition
 - Lossless Join Decomposition
 - Dependency Preserving Decomposition
- **Normalization**
 - **BCNF Decomposition Algorithm**
 - 3NF Decomposition Algorithm

Normalization

- Converting relations to BCNF or 3NF
- If a relation schema is not in BCNF
 - it is possible to obtain a lossless-join decomposition into a collection of BCNF relation schemas
 - Dependency-preserving is *not* guaranteed
- 3NF
 - There is always a dependency-preserving, lossless-join decomposition into a collection of 3NF relation schemas

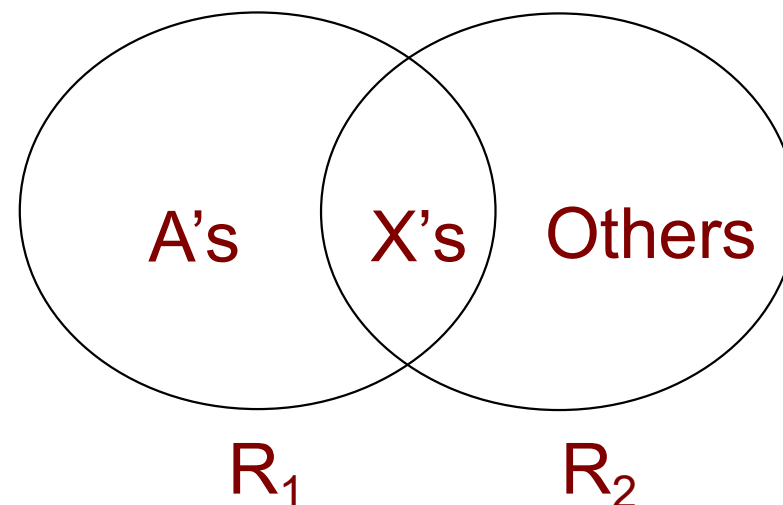
Algorithm: BCNF Decomposition

Suppose R is not in BCNF, A is an attribute, and $X \rightarrow A$ ($X_1, \dots, X_m \rightarrow A_1, \dots, A_n$) is a FD that *violates* the BCNF condition.

1. Remove A from R
2. Decompose R into XA and $R-A$

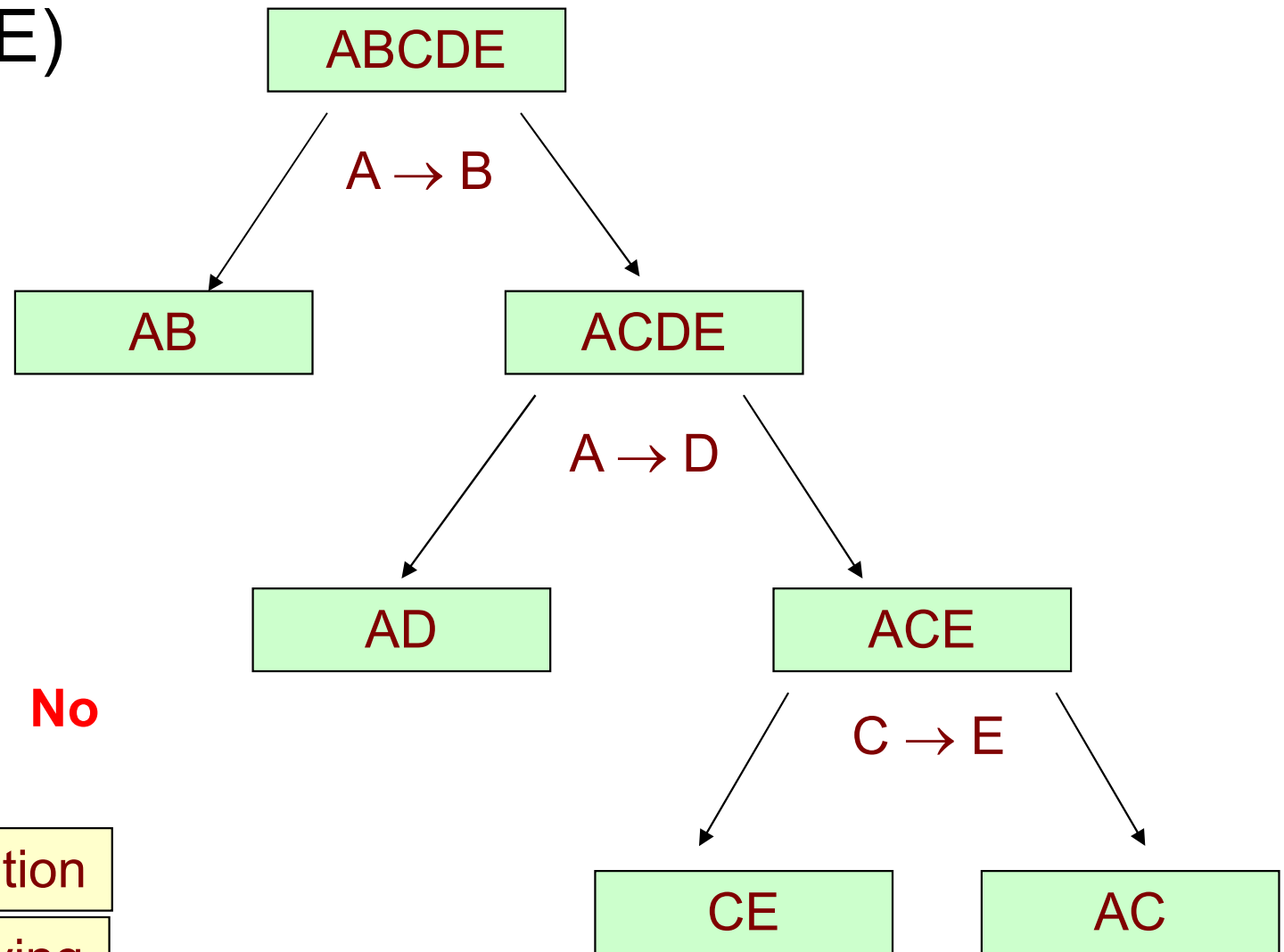
$R_1(X_1, \dots, X_m, A_1, \dots, A_n)$ and $R_2(X_1, \dots, X_m, [\text{others}])$

3. Repeat this process until all the relations become BCNF



Example: BCNF Decomposition

- $R = (A, B, C, D, E)$
- $A \rightarrow B$
- $A \rightarrow D$
- $C \rightarrow E$
- Key = ?
- Is R in BCNF? **No**

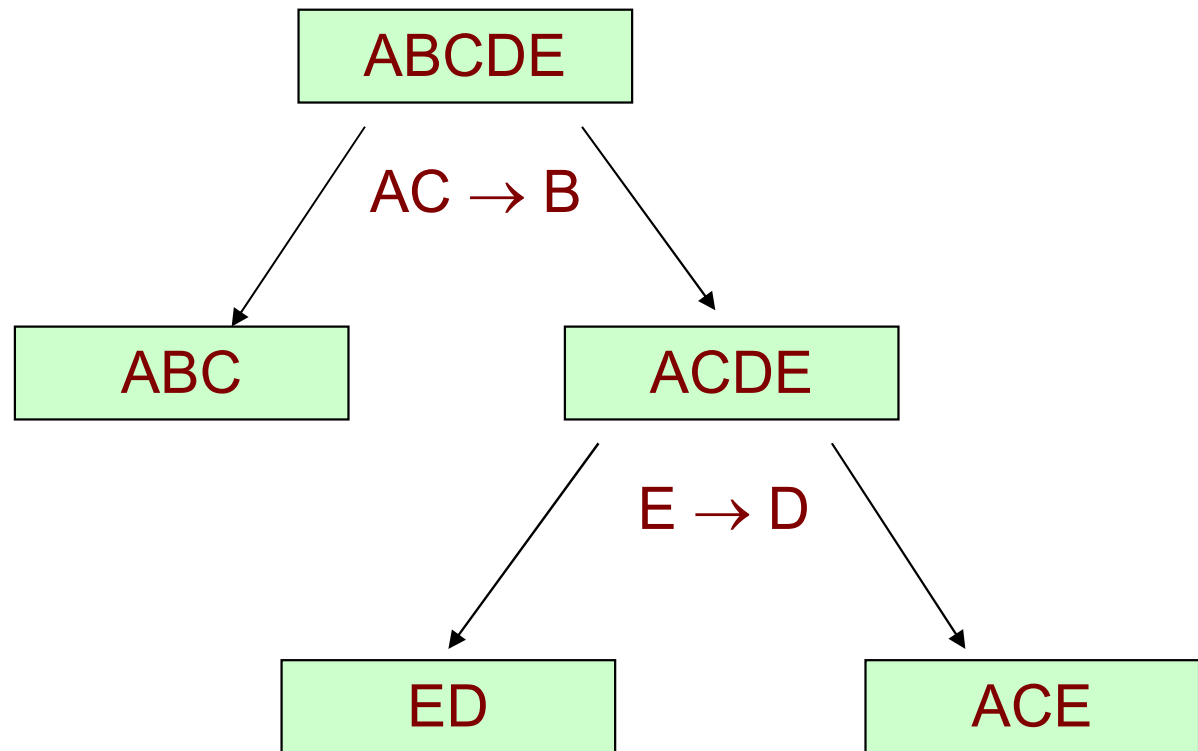


Lossless decomposition

Dependency preserving

Example: BCNF Decomposition

- $R = (A, B, C, D, E)$
- $AC \rightarrow B$
- $E \rightarrow D$
- $BE \rightarrow A$
- Key = ?
- Is R in BCNF? **No**



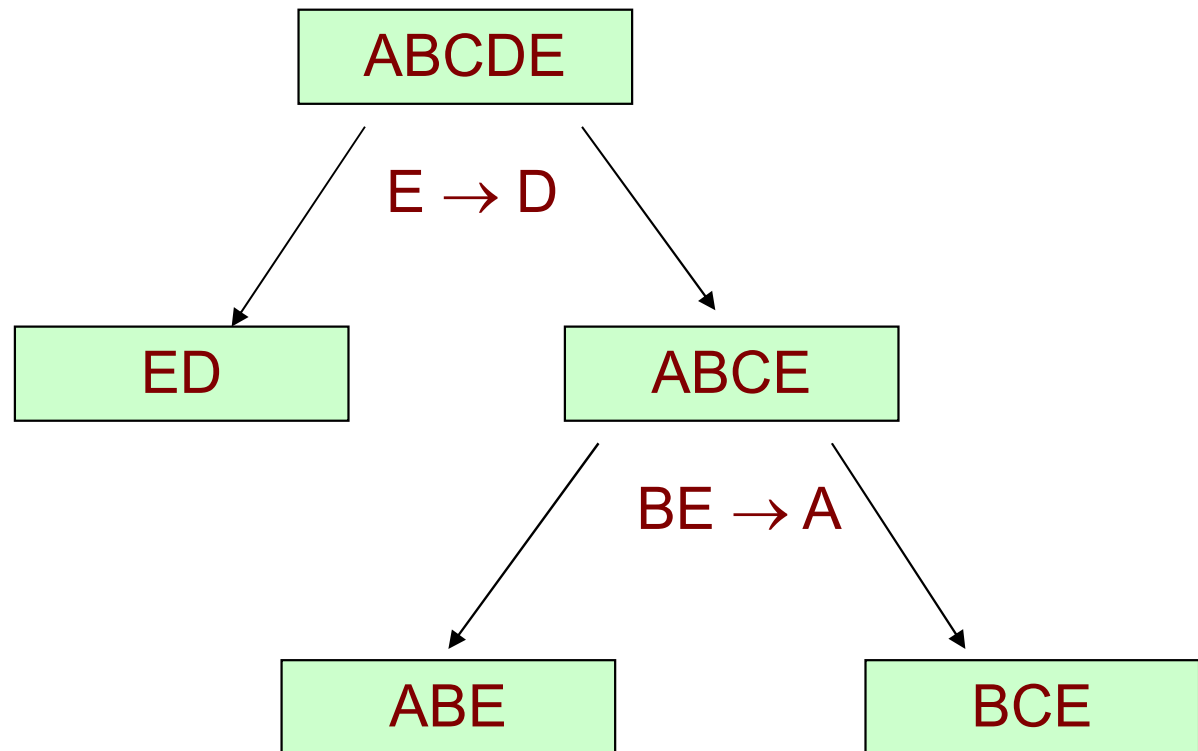
Lossless decomposition

Not Dependency preserving

Example: BCNF Decomposition

Same example

- $R = (A, B, C, D, E)$
- $AC \rightarrow B$
 $E \rightarrow D$
 $BE \rightarrow A$
- Key = ACE, BCE



Lossless decomposition

Not Dependency preserving

Different orders of chosen FDs
lead to different decompositions!

BCNF Decomposition

- BCNF decomposition guarantees that the decomposition is a *lossless-join*
- It does **not** guarantees that the decomposition is *dependency preserving*

Exercises

- Given the following two relation schemas, determine their normal form. If they are not in BCNF, decompose them into BCNF.
- $R(X, Y, Z), F = \{X \rightarrow Y, X \rightarrow Z\}$
- $R(W, X, Y, Z), F = \{X \rightarrow Y, Y \rightarrow Z, Z \rightarrow W\}$